DATA 612 · Statistical Programming in R · Week 8

8.1 dplyr: Keys and Mutating Joins

Primary keys identify rows. inner_join() and left_join() add columns from a second table.

1. Several tables, one system

One table is rarely the whole story. `nycflights13` is five tables about the same flights. Install the package once, then:

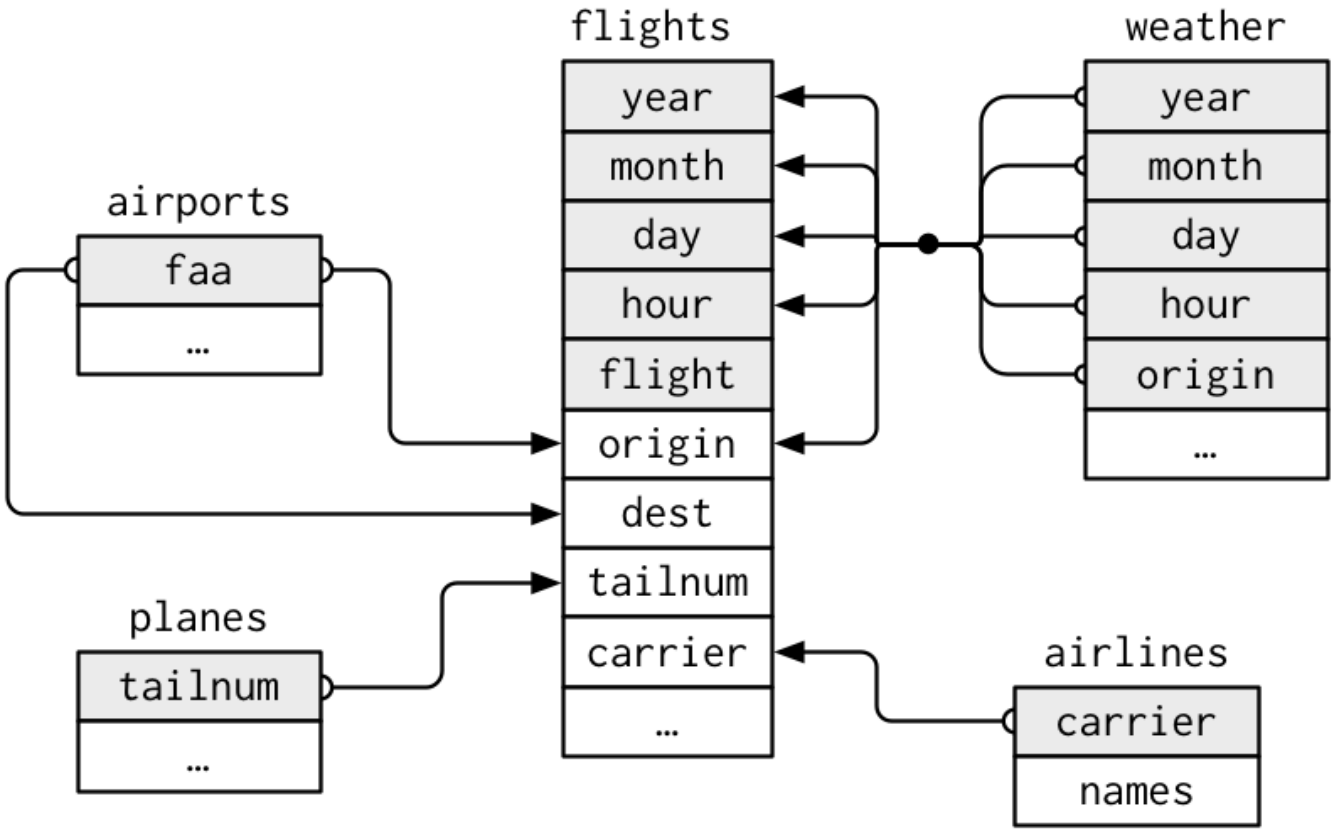
```
library(tidyverse)
library(nycflights13)

flights
planes
airlines
airports
weather
```

- `flights` — one row per flight out of NYC in 2013
- `planes` — one row per tail number
- `airlines` — carrier code and full name
- `airports` — one row per FAA code
- `weather` — hourly weather at the three NYC airports

A **primary key** uniquely identifies a row in its own table (`planes$tailnum`, `airlines$carrier`, `airports$faa`). A **foreign key** points at a primary key in another table (`flights$tailnum`, `flights$carrier`, `flights$dest`).

`weather` is identified by `origin`, `year`, `month`, `day`, and `hour` together. That is a compound key.



2. Check the keys

`count()` then `filter(n > 1)` finds duplicate keys. A real primary key has no such rows and no missing values.

```
planes |>
  count(tailnum) |>
  filter(n > 1)

weather |>
  count(origin, year, month, day, hour) |>
  filter(n > 1)

flights |>
  count(year, month, day, flight, tailnum) |>
  filter(n > 1)
```

`flights` has no natural primary key. Add a surrogate if you need one:

```
flights |>
  mutate(id = row_number())
```

3. Mutating joins add columns

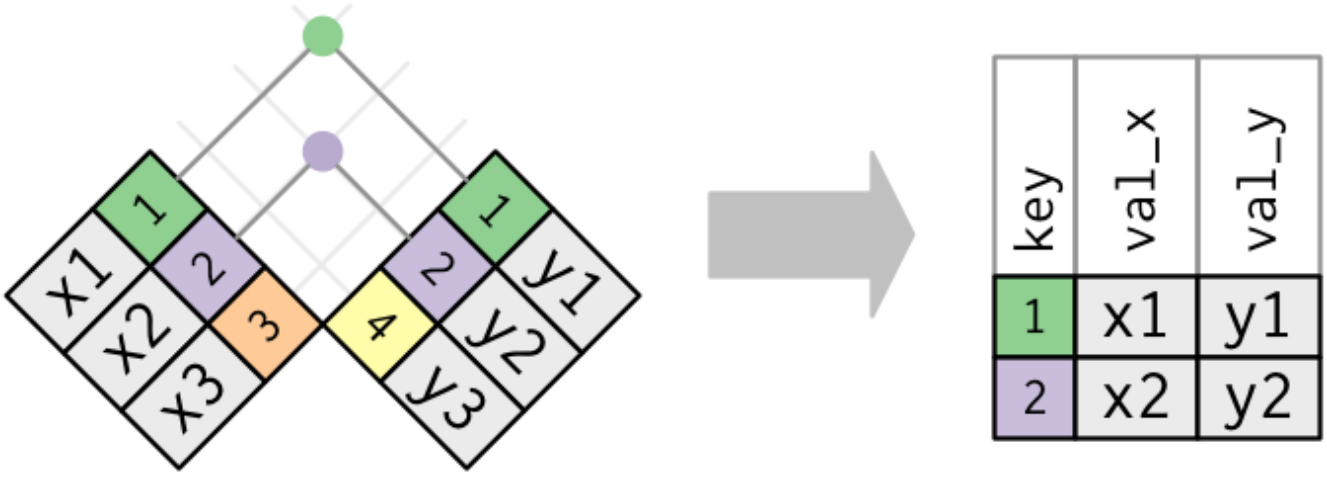
The first argument is `x`, the second is `y`. The join matches rows by key and pastes columns from `y` onto `x`.

- `inner_join(x, y)` — only keys in both
- `left_join(x, y)` — every row of `x` (the default you will use)
- `right_join(x, y)` — every row of `y`
- `full_join(x, y)` — every key in either

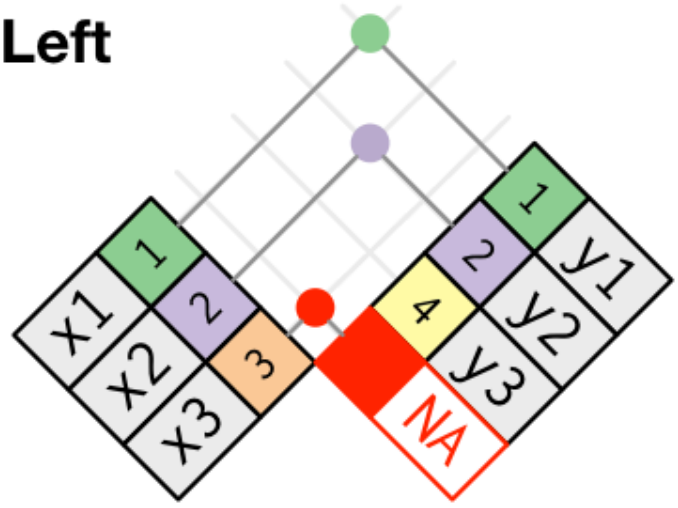
```
x <- tribble(
  ~key, ~val_x,
     1, "x1",
     2, "x2",
     3, "x3"
)
y <- tribble(
  ~key, ~val_y,
     1, "y1",
     2, "y2",
     4, "y4"
)

inner_join(x, y, by = "key")
left_join(x, y, by = "key")
right_join(x, y, by = "key")
full_join(x, y, by = "key")
```

Unmatched rows get `NA` in the new columns (`left_join` / `right_join` / `full_join`).

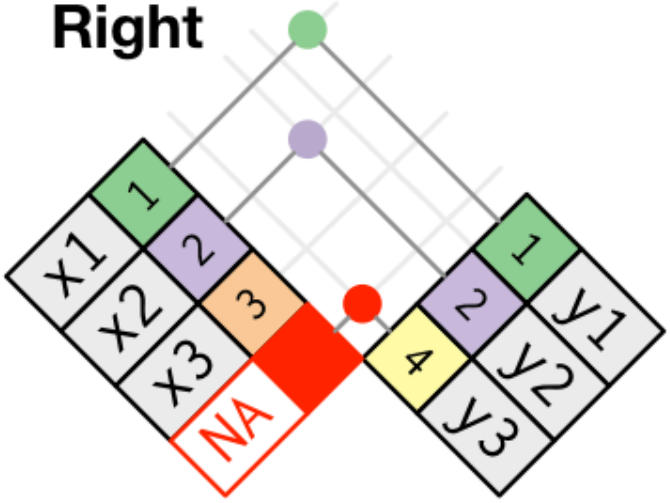


Left



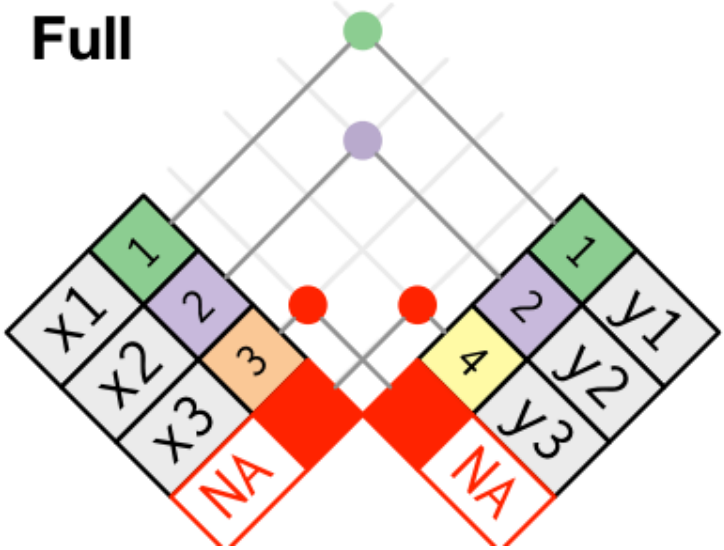
key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA

Right



key	val_x	val_y
1	x1	y1
2	x2	y2
4	NA	y3

Full



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA
4	NA	y3

`nest_join(x, y)` is the primitive: each row of `x` gets a list-column of the matching rows of `y`. The other joins are that idea plus `unnest()`. You will use `left_join()` day to day. Know `nest_join()` exists when you want the matches as a list instead of extra rows.

---

4. `by`

Same name in both tables: `by = "carrier"`. Different names: a named vector, left name first.

```
flights |>
  left_join(airlines, by = "carrier")

flights |>
  left_join(airports, by = c("dest" = "faa"))
```

Several keys: `by = c("origin", "year", "month", "day", "hour")` for weather. If you omit `by`, dplyr uses every shared name and tells you which.

When both tables have a `year` (flights and planes), the join suffixes the clash: `year.x` and `year.y`, or set `suffix`. Safer: rename before you join.

```
flights |>
  left_join(
    planes |> select(tailnum, plane_year = year),
    by = "tailnum"
  )
```

---

5. Practice

1. Add the airline name to every flight.
2. Keep only flights that have a matching row in `planes`. How many flights are dropped?
3. Name a primary key for `ggplot2::diamonds`. If there is none, say so. Optional: the same question for `Lahman::Batting` or `babynames::babynames` if you install those packages.